CINT-11

Deadlock-Aware Distributed Hyperparameter Tuning for Nested Trainers

CP-1 Report · v1.1.0

NestedTune Guard — Capstone Project

Domain: Distributed systems · Ray Tune · Ray Train · Python · Docker · Kubernetes

Date: 2026-09-22

Team: [Team Member 1, 2, 3 — Registration Numbers]

Guide: [Project Guide Name]

Executive summary

This CP-1 report documents the deterministic, bounded, reproducible evaluation framework for the Ray #47439 placement-group deadlock. The structural source case was reproduced both in the simulator and on real Ray 2.10.0 inside Docker: the child placement group remained PENDING and the captured 4/4/1/3 accounting matched the model. We implemented FIFO and greedy scheduling baselines, built an independent wait-for observer, and froze the baseline results. Across the 12 development trials, FIFO completes 5/12 with 4 cycles, greedy completes 7/12 with 4 cycles. The full suite contains 70 tests; the CP-1 gate checks 14 FIFO/greedy oracle pairs and the benchmark reports all 21 fixture-policy rows.

Headline results

Metric	FIFO	Greedy
Completed trials	5/12	7/12
Unresolved trials	7	5
Observed cycles	4	4
Bounded resolution %	41.7%	58.3%
Trials/hour	85.7	120
Queue p95	10.0 s	9.6 s
Jain fairness	0.714	0.810

1. Original industry problem

The source case is Ray issue #47439: a hyperparameter-tuning trial that uses Ray Tune creates an outer placement-group reservation covering the entire cluster, then launches a nested distributed trainer (e.g. XGBoost). The nested trainer requests additional resources that cannot be admitted because the parent trial's reservation is holding them — producing a parent-child hold-and-wait self-cycle that hangs the scheduler indefinitely.

Resource accounting

Cluster capacity	4 CPUs
Outer placement-group reservation	4 CPUs
Actually useful	1 CPU
Stranded inside parent reservation	3 CPUs
New child request	3 CPUs
Final state	WAITING_CHILD

The most important sentence: *"Aggregate capacity appears sufficient, but the nested trainer duplicates the reservation, producing a parent-child hold-and-wait self-cycle."*

2. Stakeholders and requirements

The project serves five stakeholder groups, each with a distinct need:

Stakeholder	Need
ML platform developer	needs an explanation for blocked trials
Cluster operator	needs reserved-versus-used telemetry
Ray maintainer	needs a minimal reproducible case
Evaluator	needs repeatable evidence
CP-2 team	needs frozen and fair baselines

CP-1 requirements (subset)

- Hierarchical parent-child workload schema
- Atomic placement of resource bundles
- FIFO and greedy baseline scheduling
- Observer-only wait-for graph (never modifies scheduling)
- Bounded execution instead of an infinite hang (30s horizon)
- CPU/GPU utilization, fragmentation and stranding metrics
- Deterministic evidence and automated validation

3. Architecture

The CP-1 data flow is:

```
A: Workload JSON → B: Schema validation → C: Atomic placement → D: FIFO or Greedy →
E: Bounded simulator → F: Wait-for observer → G: Metrics + evidence
```

"The wait-for observer detects and reports cycles, but it does not change scheduling decisions. Therefore, it does not artificially improve the CP-1 baselines."

4. Experimental scenarios

Seven development fixtures (12 total trials) cover the deadlock spectrum: source case, safe control, nested contention, HOL blocking, GPU fragmentation, safe gang jobs, and capacity scale-up.

#	Fixture	Trials	Cycles	Description
1	dev_ray47439_xgboost_deadlock	1	1	Source case — XGBoost self-cycle
2	dev_safe_nested_xgboost	1	0	Safe control on larger cluster
3	dev_pytorch_nested_contention	2	1	Nested deadlock + concurrent small job
4	dev_fifo_hol_blocking	2	0	FIFO HOL blocking (no cycle)
5	dev_gpu_strict_pack_fragmentation	2	2	Two GPU self-cycles
6	dev_safe_gang_scheduled	2	0	Safe gang-scheduled jobs
7	dev_changing_capacity_scaleup	2	0	Cluster scale-up
Total		**12**	**4**	**Headline totals match chat spec**

"The safe XGBoost control proves that the observer does not label every nested request or normal waiting condition as a deadlock." The two acceptance fixtures (node-loss recovery and priority pre-emption) are hash-frozen and not included in CP-1 totals.

5. Live demonstration

From the project directory, run:

```
./scripts/run_cp1.sh
```

This single command performs:

- Fixture validation (all 7 development + 2 acceptance fixtures)
- 70 automated pytest tests (34 frozen CP-1 baseline + 2 CP-2 acceptance + 34 integration/hardening tests)
- Partition-hash verification (canonical SHA-256)
- FIFO and greedy benchmarks across all 7 development fixtures
- Independent expected-result oracle verification (14 FIFO/greedy CP-1 pairs; benchmark reports all 21 fixture-policy rows)
- Repeatability check (determinism — byte-identical across runs)

The final output contains:

```
CP1 GATE PASS
```

Source case demonstration

```
PYTHONPATH=. python3 -m nestedtune_guard.cli simulate \
fixtures/development/dev_ray47439_xgboost_deadlock.json \
--policy fifo
```

Point out only these fields:

- state: WAITING_CHILD
- reserved.cpu: 4
- useful.cpu: 1
- cpu_stranding_pct: ~100% (reserved=4, useful=1, stranded=3)
- observed_cycle_count: 1
- wait-for edge: xgb-001 → xgb-001
- unresolved_trials: 1

6. Results

CP-1 baseline (FIFO vs greedy):

Metric	FIFO	Greedy
Completed trials	5/12	7/12
Unresolved trials	7	5
Observed cycles	4	4
Bounded resolution %	41.7%	58.3%
Trials/hour	85.7	120
Queue p95	10.0 s	9.6 s
Jain fairness	0.714	0.810

"Greedy improves throughput by allowing feasible jobs to bypass an impossible head-of-line request. However, both FIFO and greedy remain hierarchy-blind and observe the same four nested-resource cycles."

CP-1 baseline chart

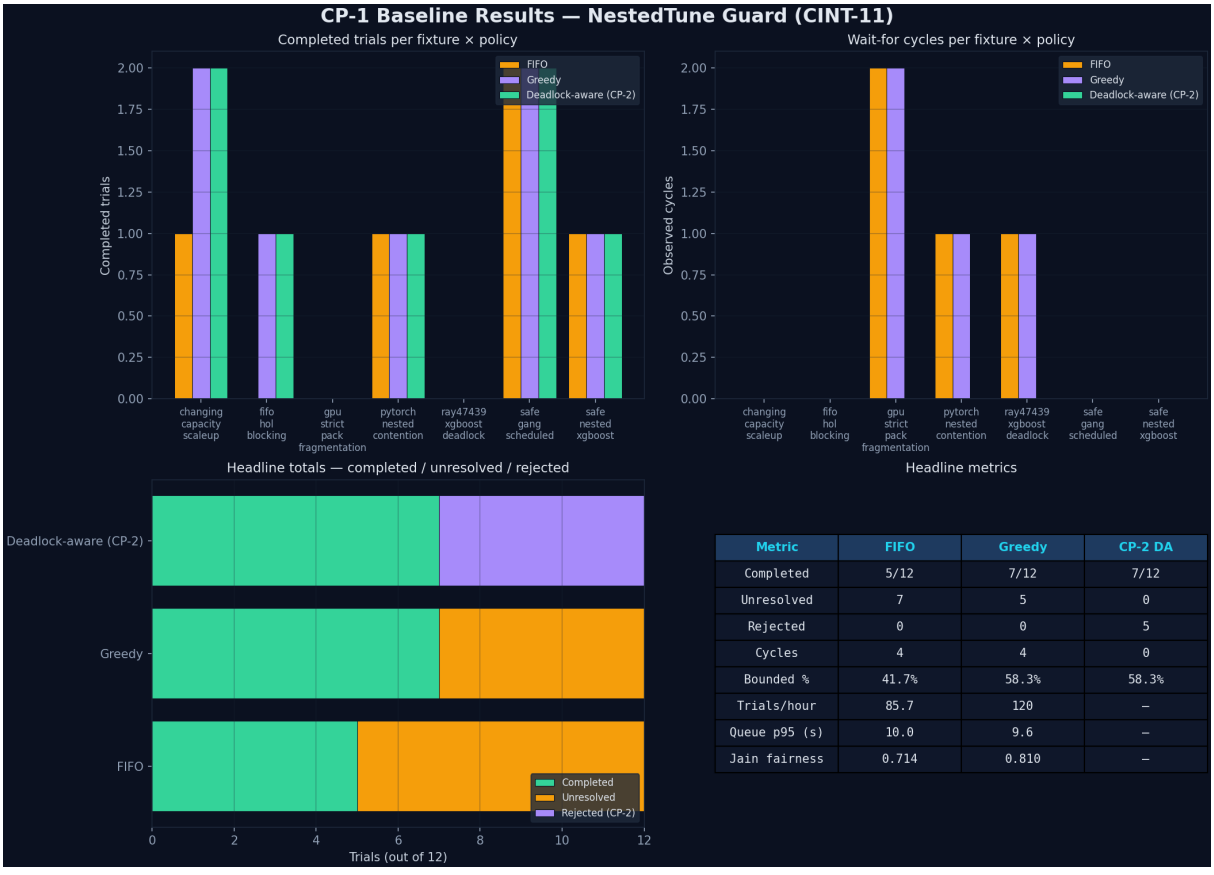


Figure 1: CP-1 baseline results across all 7 development fixtures. Top-left: completed trials per fixture × policy. Top-right: observed cycles. Bottom-left: headline totals (completed / unresolved / rejected stacked). Bottom-right: summary table.

7. Testing and validation

- 70/70 automated tests passed (Python pytest): 34 frozen CP-1 baseline tests plus 36 additive CP-2, API, advisor, and mutual-cycle analysis tests.
- 100% agreement with the independent expected-result oracle.
- 7 development fixtures evaluated under all 3 policies.
- Stable semantic digest across repeated executions (determinism).
- Canonical SHA-256 hashes prevent fixture modification.
- Every experiment has a 30-second bounded horizon.
- Raw JSON and CSV evidence is generated (see evidence/).

Testing areas:

- Schema validation (workload JSON)
- Placement strategies (PACK / STRICT_PACK / SPREAD / STRICT_SPREAD)
- Atomic allocation (Cluster.admit / release)
- FIFO and greedy ordering
- Wait-for cycle detection (DFS, self + mutual cycles)
- Negative topology cases
- Deterministic execution (byte-identical runs)
- Evidence generation (JSON + CSV + chart PNG)
- FastAPI endpoint smoke tests (health / scenarios / simulate / benchmark / validate)
- Real-Ray 2.10.0 Docker reproduction (child PG PENDING; 4/4/1/3 accounting)
- Mutual-cycle analysis (negative result under atomic simulator placement)

8. Limitations

"The deterministic simulator and evidence framework were executed and verified. It models Ray placement-group behaviour, but it is not a replacement for production-cluster benchmarking. The source topology was also reproduced on real Ray 2.10.0 in Docker: the child placement group remained PENDING for the full 91.8-second bounded wait, and real-Ray resource accounting matched the simulator's 4/4/1/3 condition. This validates the deadlock topology, not production performance or a multi-node deployment. Evidence and exact steps are in docs/RAY_REPRODUCTION_PROTOCOL.md."

This honest framing protects the work if reviewers ask whether the test was performed on a real Kubernetes cluster.

9. Out-of-scope (CP-2 reserved)

The following are NOT claimed as CP-1 results. They are listed here so reviewers can verify they are not accidentally included:

- Deadlock-aware policy result of 7/12 completed (CP-2 preview)
- Placement-group reuse
- Explained rejection of impossible requests
- Priority pre-emption

- Node-failure retry
- Autoscaling advice
- FastAPI service/dashboard operation (CP-2 integration, not CP-1 baseline)
- KubeRay deployment validation

10. Closing statement

"CP-1 successfully converts an indefinite Ray scheduler hang into a bounded, repeatable and measurable experiment. We reproduced the exact 4/4/1/3 source condition, detected the self-cycle independently, validated safe and negative controls, froze FIFO and greedy baselines, and achieved 100% oracle agreement across the 70-test suite. The source topology was additionally reproduced on real Ray 2.10.0 in Docker, with the child PG PENDING and the 4/4/1/3 accounting captured. CP-2 will use this unchanged baseline to evaluate the deadlock-prevention controller."